

Wallet Application Security Audit Report

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

3.3 Vulnerability Summary

4 Audit Result

5 Statement

1 Executive Summary

On 2025.03.27, the SlowMist security team received the Haust Lab team's security audit application for Haust Wallet, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "black-box and grey-box" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for web wallet includes two steps:

The codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The web wallets are manually analyzed to look for any potential issues.

The following is a list of security audit items considered during an audit:

- Transfer security
    - Signature security audit
    - Deposit/Transfer security audit
    - Transaction broadcast security audit
- Secret key security
    - Secret key generation security audit
    - Secret key storage security audit
    - Secret key usage security audit
    - Secret key backup security audit
    - Secret key destruction security audit
    - Random generator security audit
    - Cryptography security audit
- Web front-end security
    - Cross-Site Scripting security audit
    - Third-party JS security audit
    - HTTP response header security audit
    - DNSSEC security audit
    - SSL/TLS security audit
- Communication security
    - Communication encryption security audit
    - Cross-domain transmission security audit

- Architecture and business logic security
    - Access control security audit
    - Wallet lock security audit
    - Business design security audit
    - Architecture design security audit
    - Denial of Service security audit
- User interaction security

3 Project Overview

3.1 Project Introduction

Audit Version

Web Site

https://wallet.wdev.haust.app

Wallet Frontend

Project Address: https://github.com/Haust-Labs/haust-wallet-frontend

Commit: f5b28b939aaae1dc09488ea129156ef98b74e1e3

Wallet Backend

Project Address: https://github.com/Haust-Labs/haust-wallet-rs

Commit: 7ea6d8ef3ca0af28b4b294b830c5edac6b3772a2

Fixed Version

Web Site

https://wallet.stage.haust.app

Wallet Frontend

Project Address: https://github.com/Haust-Labs/haust-wallet-frontend

Commit: b0220f46d2268108e4b4c5652045e9818a6a2fae

Wallet Backend

Project Address: https://github.com/Haust-Labs/haust-wallet-rs

Commit: d18b420ac1691b71072e9eb6e9d1c266c6ba446c

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Signature content issue	Signature security audit	Suggestion	Fixed
N2	Transfer fee issue	Deposit/Transfer security audit	High	Fixed
N3	Clickjacking protection security Issue	HTTP response header security audit	Medium	Fixed
N4	Lack of Content-Security-Policy security configuration	HTTP response header security audit	Suggestion	Fixed
N5	Lack of X-Content-Type-Options security configuration	HTTP response header security audit	Suggestion	Fixed
N6	Cross-domain transmission security issue	Cross-domain transmission security audit	Suggestion	Fixed
N7	WalletConnect issue	Business design security audit	Low	Fixed
N8	Email binding issue	Business design security audit	Suggestion	Fixed
N9	Denial of Service security issue	Denial of Service security audit	Low	Fixed
N10	DNSSEC security issue	DNSSEC security audit	Suggestion	Fixed
N11	User interaction issue	User interaction security	Suggestion	Acknowledged
N12	Debug information is printed to console log by default	Others	Suggestion	Fixed

NO	Title	Category	Level	Status
N13	API error exposes detailed stack trace leading to technology stack disclosure	Others	Low	Fixed
N14	Signatures can be reused multiple times	Others	Medium	Fixed
N15	Insufficient authentication security due to missing JWT claims	Others	Low	Fixed
N16	Incomplete token validation	Others	Low	Fixed
N17	Arbitrary email spoofing due to SPF configuration errors	Others	Low	Fixed
N18	Lack of anti-phishing security code	Others	Suggestion	Fixed

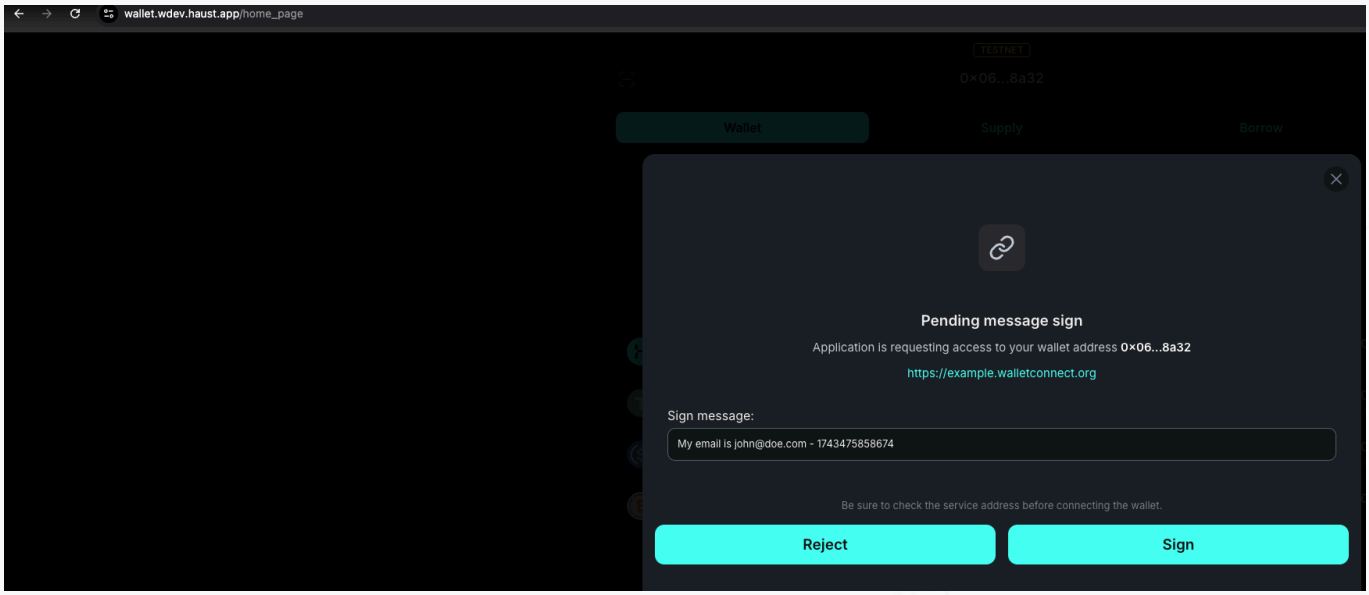
3.3 Vulnerability Summary

[N1] [Suggestion] Signature content issue

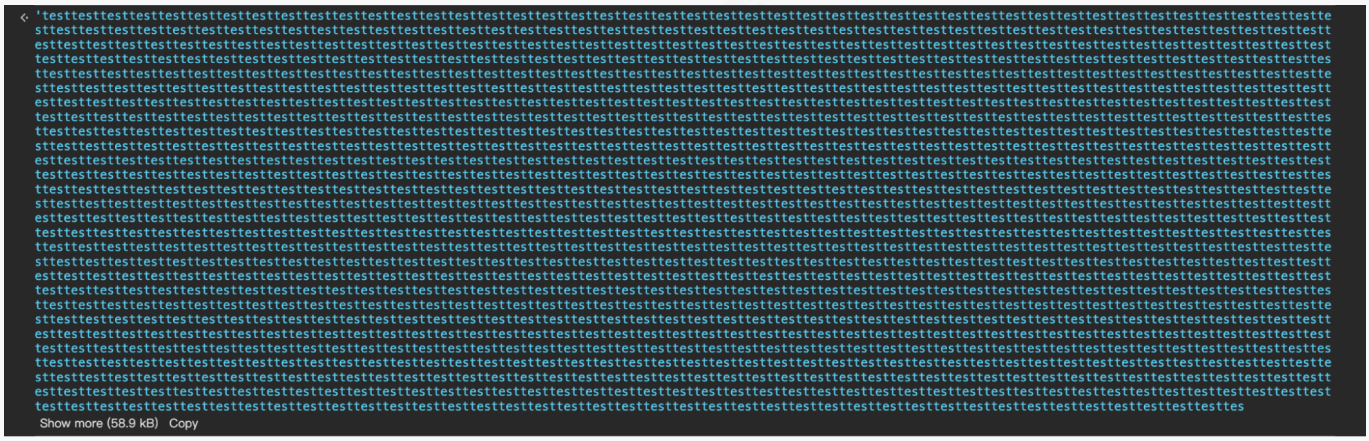
Category: Signature security audit

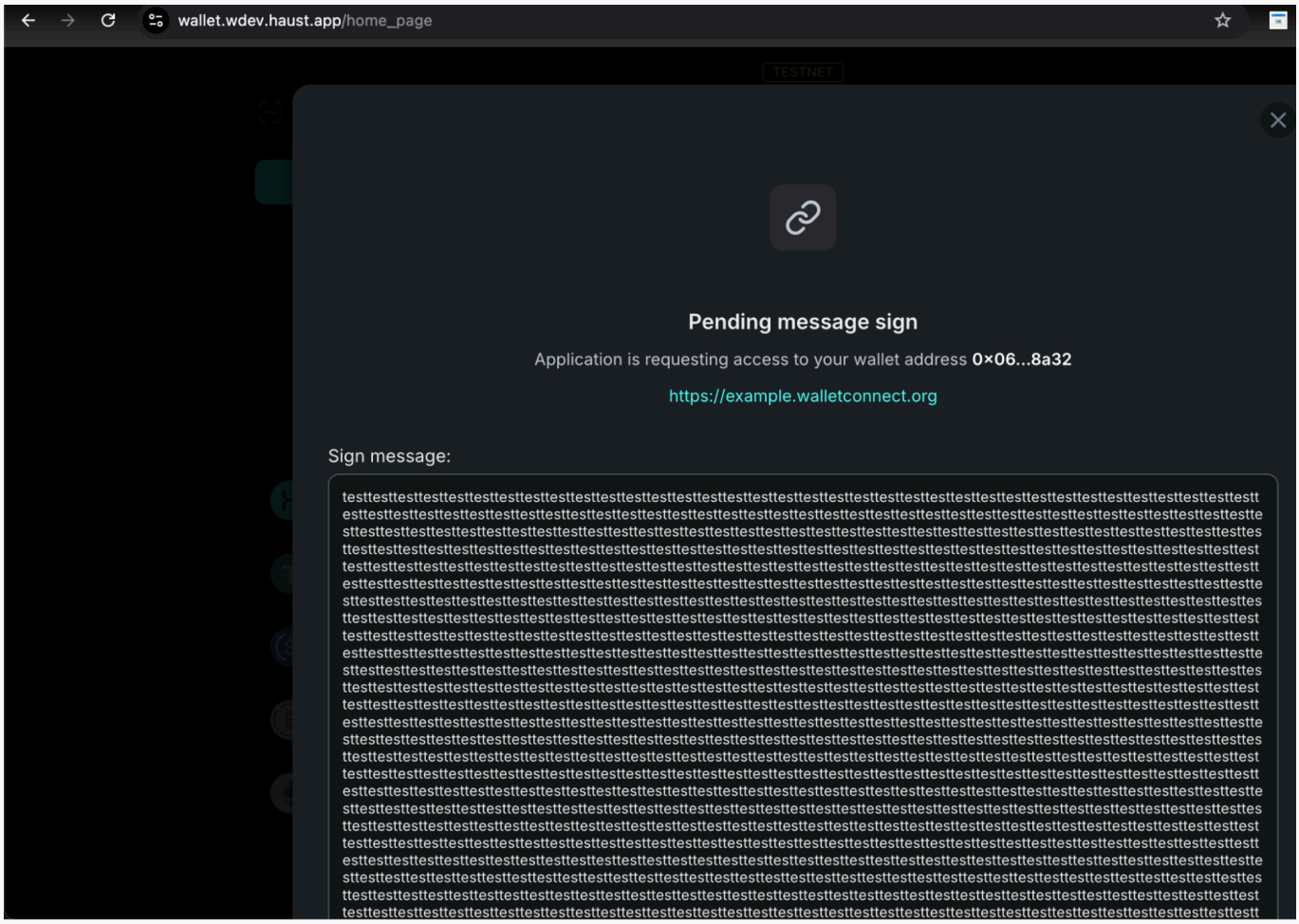
Content

When signing, the wallet will present the signature source to the user and display the parsed signature content for the user.



A long signature will cause the wallet to crash. For example, initiate a signature using the following signature content:


'testtesttesttestt



Solution

It is recommended to incorporate a drop-down box to fully exhibit signature data when the signature is overly long.

Status

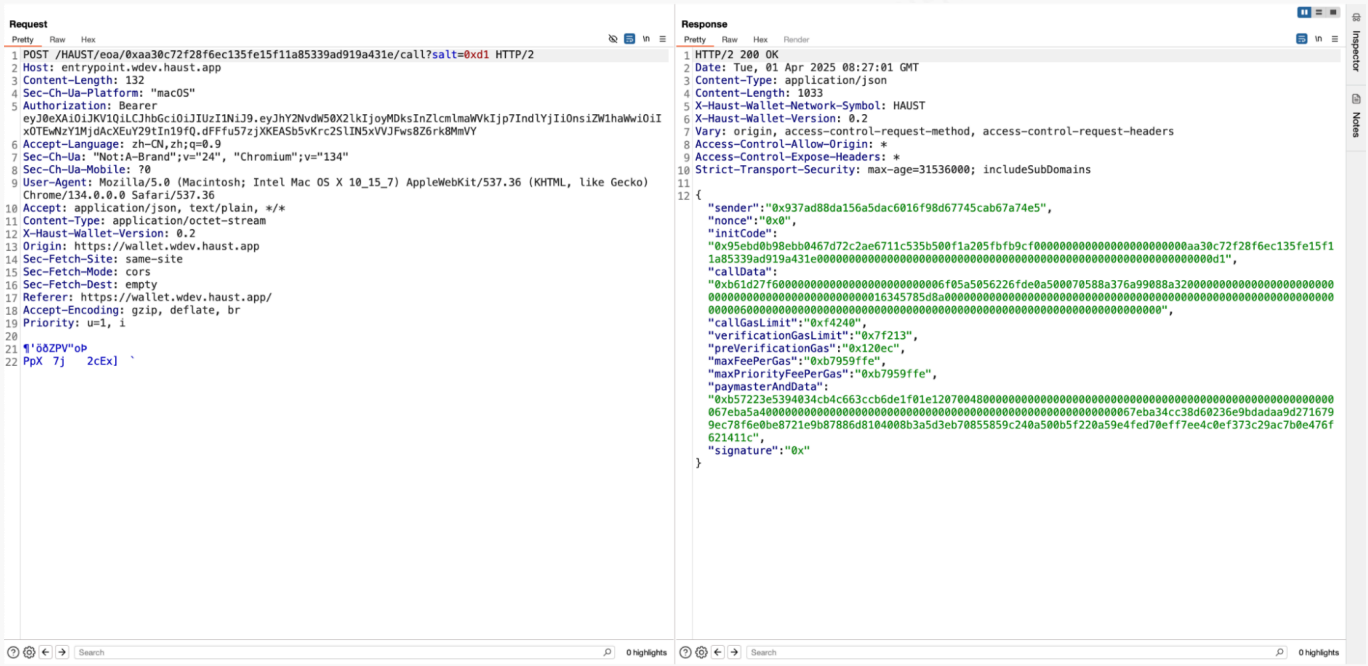
Fixed

[N2] [High] Transfer fee issue

Category: Deposit/Transfer security audit

Content

When conducting a transfer, initially request transfer data from the backend.



Request

```
POST /HAUST/eoa/0xaa30c72f28f6ec135fe15f11a85339ad919a431e/call?salt=0xd1 HTTP/2
Host: entrypoint.wdev.haust.app
Content-Length: 132
Sec-Ch-Ua-Platform: "macOS"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhY2NvdW50X2lkIjoyMDksInZlcmlmaWVkIjp7IndlYjIiOnsiZW1haWwiOiI
xOTEwNzY1MjdAcXEuY29tIn19fQ.dFFfu57zjXKEASb5vKrc2SlIN5xVVJFws8Z6rk8MmVY
Accept-Language: zh-CN,zh;q=0.9
Sec-Ch-Ua: "Not:A-Brand";v="24", "Chromium";v="134"
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/134.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/octet-stream
X-Haust-Wallet-Version: 0.2
Origin: https://wallet.wdev.haust.app
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://wallet.wdev.haust.app/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

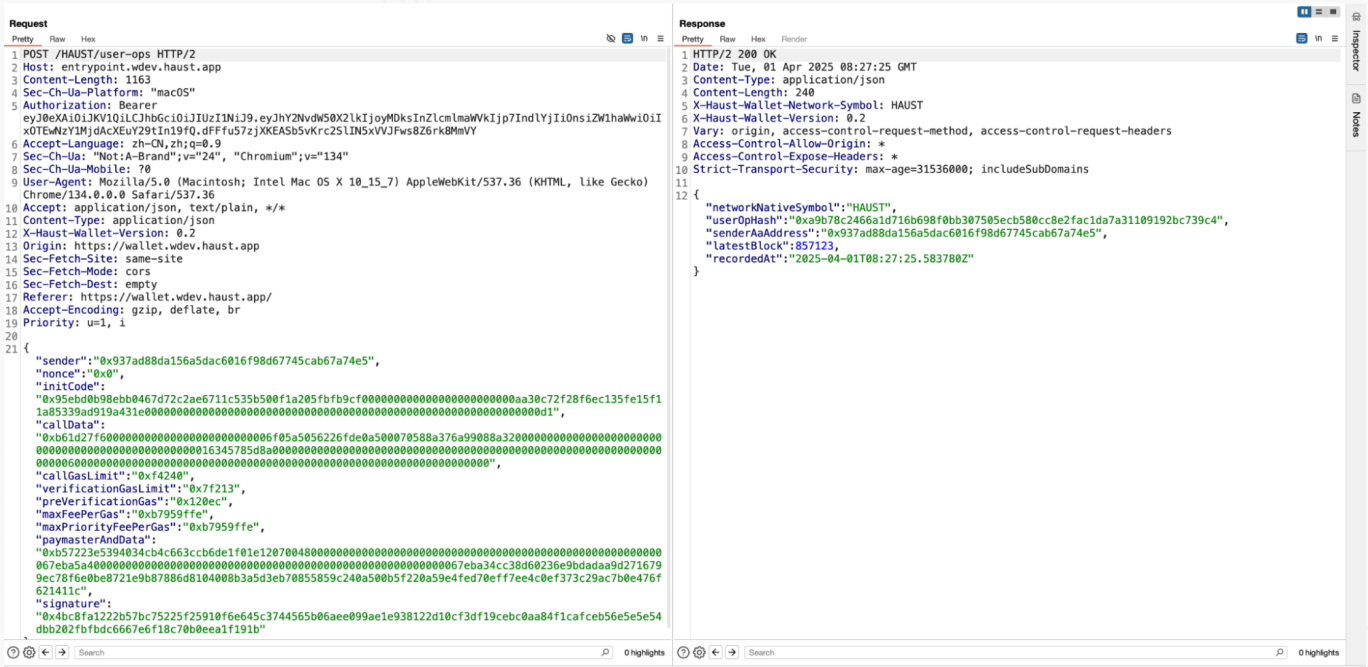
¶'ôðZPV"oÞ
PpX 7j  2cEx] `
```

Response

```
HTTP/2 200 OK
Date: Tue, 01 Apr 2025 08:27:01 GMT
Content-Type: application/json
Content-Length: 1033
X-Haust-Wallet-Network-Symbol: HAUST
X-Haust-Wallet-Version: 0.2
Vary: origin, access-control-request-method, access-control-request-headers
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: *
Strict-Transport-Security: max-age=31536000; includeSubDomains

{
  "sender":"0x937ad88da156a5dac6016f98d67745cab67a74e5",
  "nonce":"0x0",
  "initCode":
  "0x95ebd0b98ebb0467d72c2ae6711c535b500f1a205fbfb9cf0000000000000000000000000aa30c72f28f6ec135fe15f1
  1a85339ad919a431e0000000000000000000000000000000000000000000000000000000000000000d1",
  "callData":
  "0xb61d27f6000000000000000000000000006f05a5056226fde0a500070588a376a99088a32000000000000000000000000
  0000000000000000000000000016345785d8a0000000000000000000000000000000000000000000000000000000000000
  0000060000000000000000000000000000000000000000000000000000000000000000",
  "callGasLimit":"0xf4240",
  "verificationGasLimit":"0x7f213",
  "preVerificationGas":"0x120ec",
  "maxFeePerGas":"0xb7959ffe",
  "maxPriorityFeePerGas":"0xb7959ffe",
  "paymasterAndData":
  "0xb57223e5394034cb4c663ccb6de1f01e12070048000000000000000000000000000000000000000000000000000000000000
  067eba5a4000000000000000000000000000000000000000000000000000000000000000067eba34cc38d60236e9bdadaa9d271679
  9ec78f6e0be8721e9b87886d8104008b3a5d3eb70855859c240a500b5f220a59e4fed70eff7ee4c0ef373c29ac7b0e476f
  621411c",
  "signature":"0x"
}
```

Upon obtaining the data, perform signing and then transmit it to the backend.



Request

```
POST /HAUST/user-ops HTTP/2
Host: entrypoint.wdev.haust.app
Content-Length: 1163
Sec-Ch-Ua-Platform: "macOS"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhY2NvdW50X2lkIjoyMDksInZlcmlmaWVkIjp7IndlYjIiOnsiZW1haWwiOiI
xOTEwNzY1MjdAcXEuY29tIn19fQ.dFFfu57zjXKEASb5vKrc2SlIN5xVVJFws8Z6rk8MmVY
Accept-Language: zh-CN,zh;q=0.9
Sec-Ch-Ua: "Not:A-Brand";v="24", "Chromium";v="134"
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/134.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
X-Haust-Wallet-Version: 0.2
Origin: https://wallet.wdev.haust.app
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://wallet.wdev.haust.app/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

{
  "sender":"0x937ad88da156a5dac6016f98d67745cab67a74e5",
  "nonce":"0x0",
  "initCode":
  "0x95ebd0b98ebb0467d72c2ae6711c535b500f1a205fbfb9cf0000000000000000000000000aa30c72f28f6ec135fe15f1
  1a85339ad919a431e0000000000000000000000000000000000000000000000000000000000000000d1",
  "callData":
  "0xb61d27f6000000000000000000000000006f05a5056226fde0a500070588a376a99088a32000000000000000000000000
  0000000000000000000000000016345785d8a0000000000000000000000000000000000000000000000000000000000000
  0000060000000000000000000000000000000000000000000000000000000000000000",
  "callGasLimit":"0xf4240",
  "verificationGasLimit":"0x7f213",
  "preVerificationGas":"0x120ec",
  "maxFeePerGas":"0xb7959ffe",
  "maxPriorityFeePerGas":"0xb7959ffe",
  "paymasterAndData":
  "0xb57223e5394034cb4c663ccb6de1f01e12070048000000000000000000000000000000000000000000000000000000000000
  067eba5a4000000000000000000000000000000000000000000000000000000000000000067eba34cc38d60236e9bdadaa9d271679
  9ec78f6e0be8721e9b87886d8104008b3a5d3eb70855859c240a500b5f220a59e4fed70eff7ee4c0ef373c29ac7b0e476f
  621411c",
  "signature":
  "0x4bc8fa1222b57bc75225f25910f6e645c3744565b06aee099ae1e938122d10cf3df19cebc0aa84f1cafceb56e5e5e54
  dbb202fbfbdc6667e6f18c70b0eea1f191b"
}
```

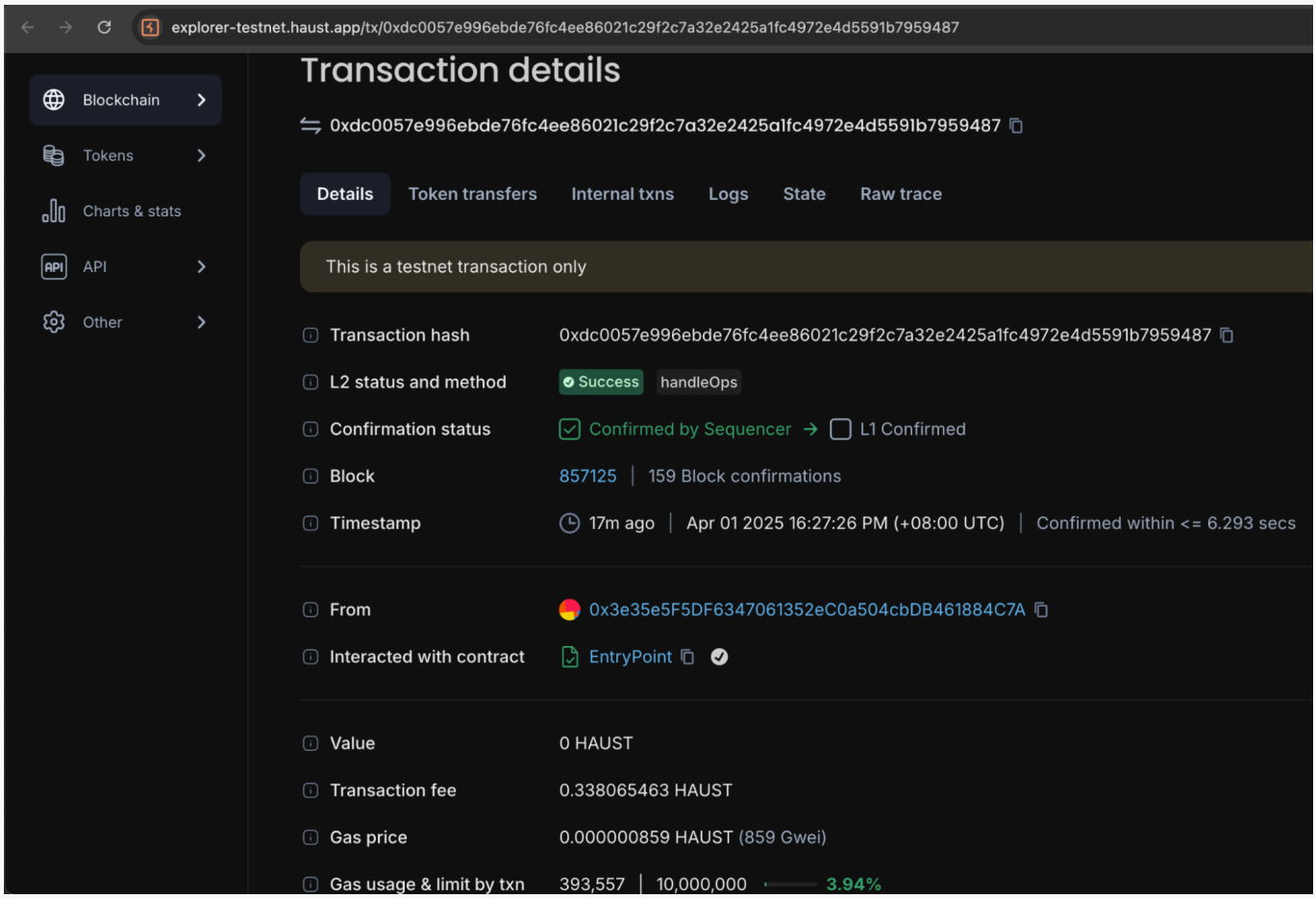
Response

```
HTTP/2 200 OK
Date: Tue, 01 Apr 2025 08:27:25 GMT
Content-Type: application/json
Content-Length: 240
X-Haust-Wallet-Network-Symbol: HAUST
X-Haust-Wallet-Version: 0.2
Vary: origin, access-control-request-method, access-control-request-headers
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: *
Strict-Transport-Security: max-age=31536000; includeSubDomains

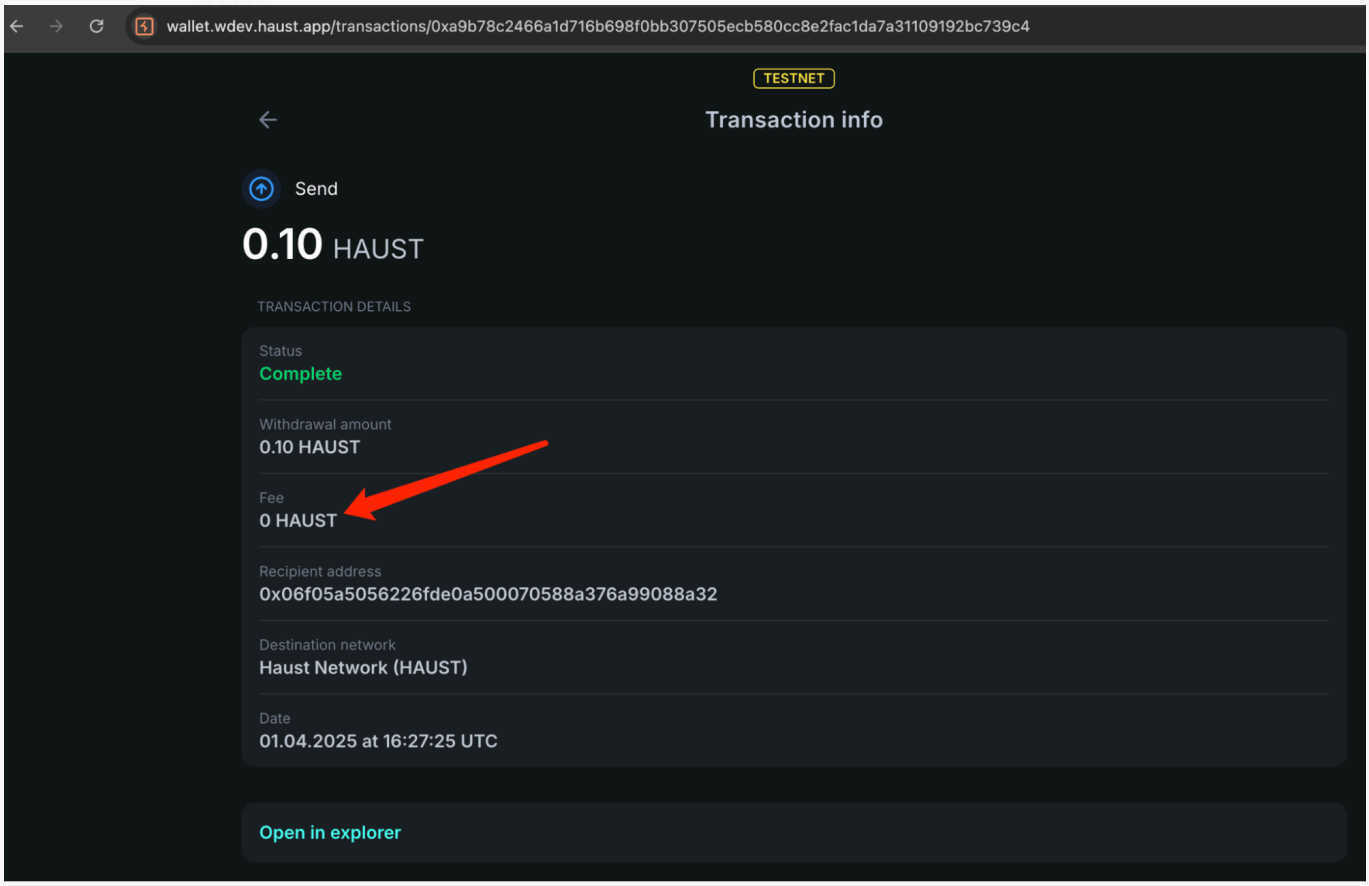
{
  "networkNativeSymbol":"HAUST",
  "userOpHash":"0xa9b78c2466a1d716b698f0bb307505ecb580cc8e2fac1da7a31109192bc739c4",
  "senderAaAddress":"0x937ad88da156a5dac6016f98d67745cab67a74e5",
  "latestBlock":857123,
  "recordedAt":"2025-04-01T08:27:25.583780Z"
}
```

After the back end acquires the data, it conducts signature verification. Once the signature verification is successful,

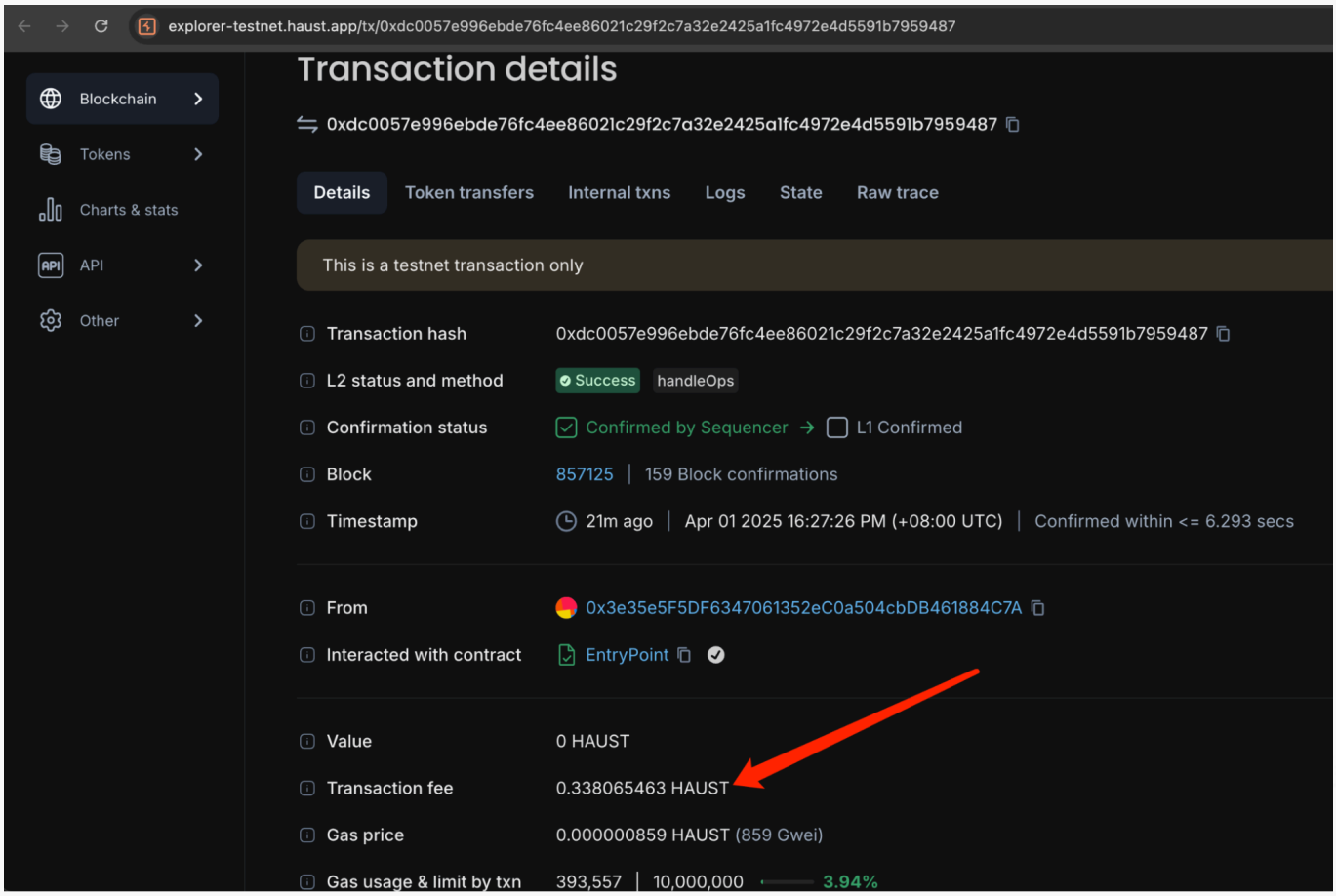
the data is uploaded to the blockchain by the address 0x3e35e5F5DF6347061352eC0a504cbDB461884C7A.



In the test, it was discovered that the handling fee for users to send tokens is zero.



However, the handling fee for being on the chain is 0.338065463 HAUST.



If an attacker maliciously initiates multiple transfers, it will result in the malicious consumption of tokens in the address 0x3e35e5F5DF6347061352eC0a504cbDB461884C7A.

Solution

It is recommended to charge appropriate handling fees when conducting transfers.

Status

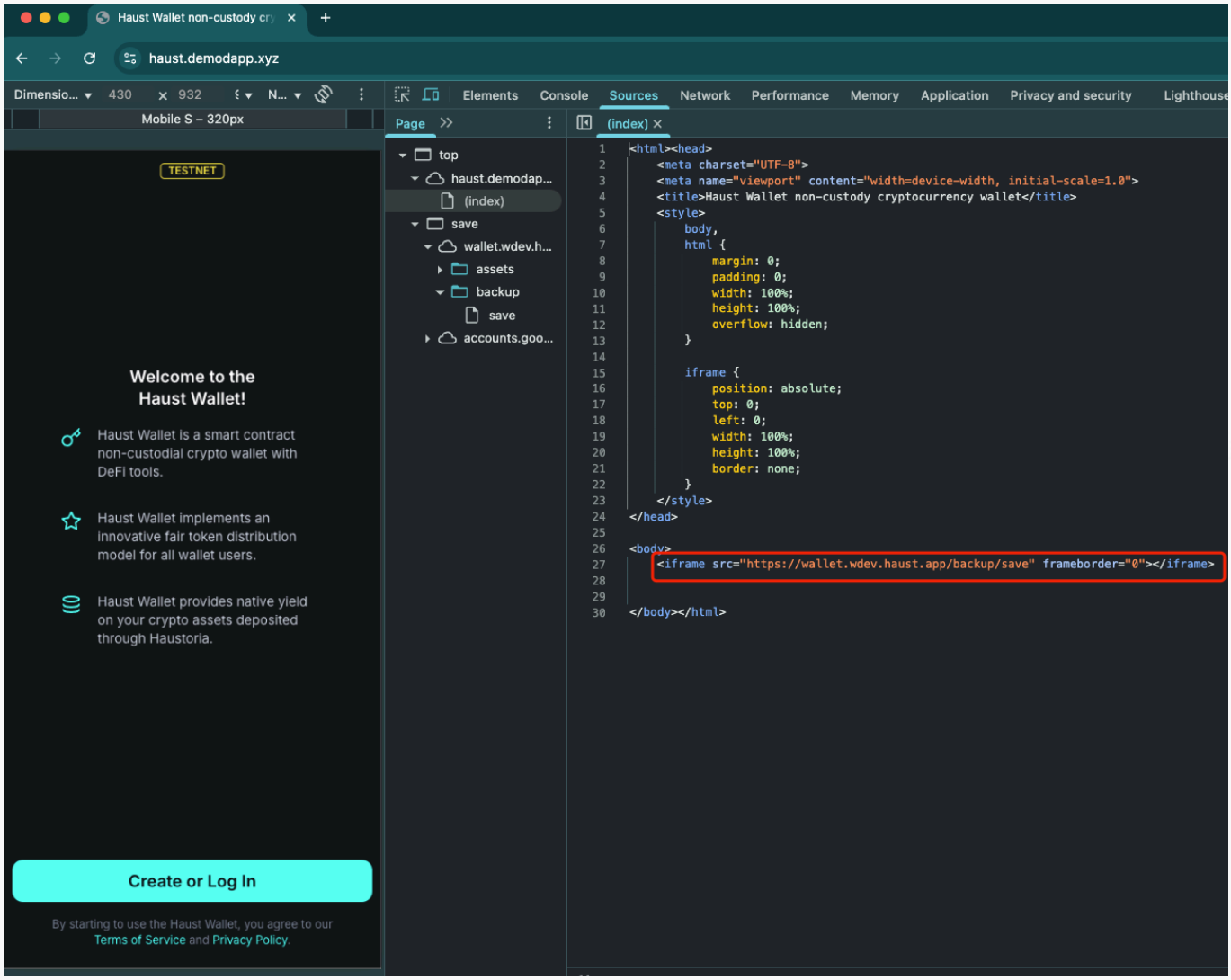
Fixed; At this stage, only the handling fees for the first 10 transactions are permitted to be waived.

[N3] [Medium] Clickjacking protection security Issue

Category: HTTP response header security audit

Content

The site lacks the X-FRAME-OPTIONS security configuration, which allows the page to be embedded in an iframe tag, which can be used for phishing.



Solution

The essence of countering Clickjacking is to counteract that your own service is embedded in the iframe/frame method of pages of other domains.

The HTTP response header configuration:

```
X-FRAME-OPTIONS: SAMEORIGIN
```

or

```
X-FRAME-OPTIONS: DENY
```

Status

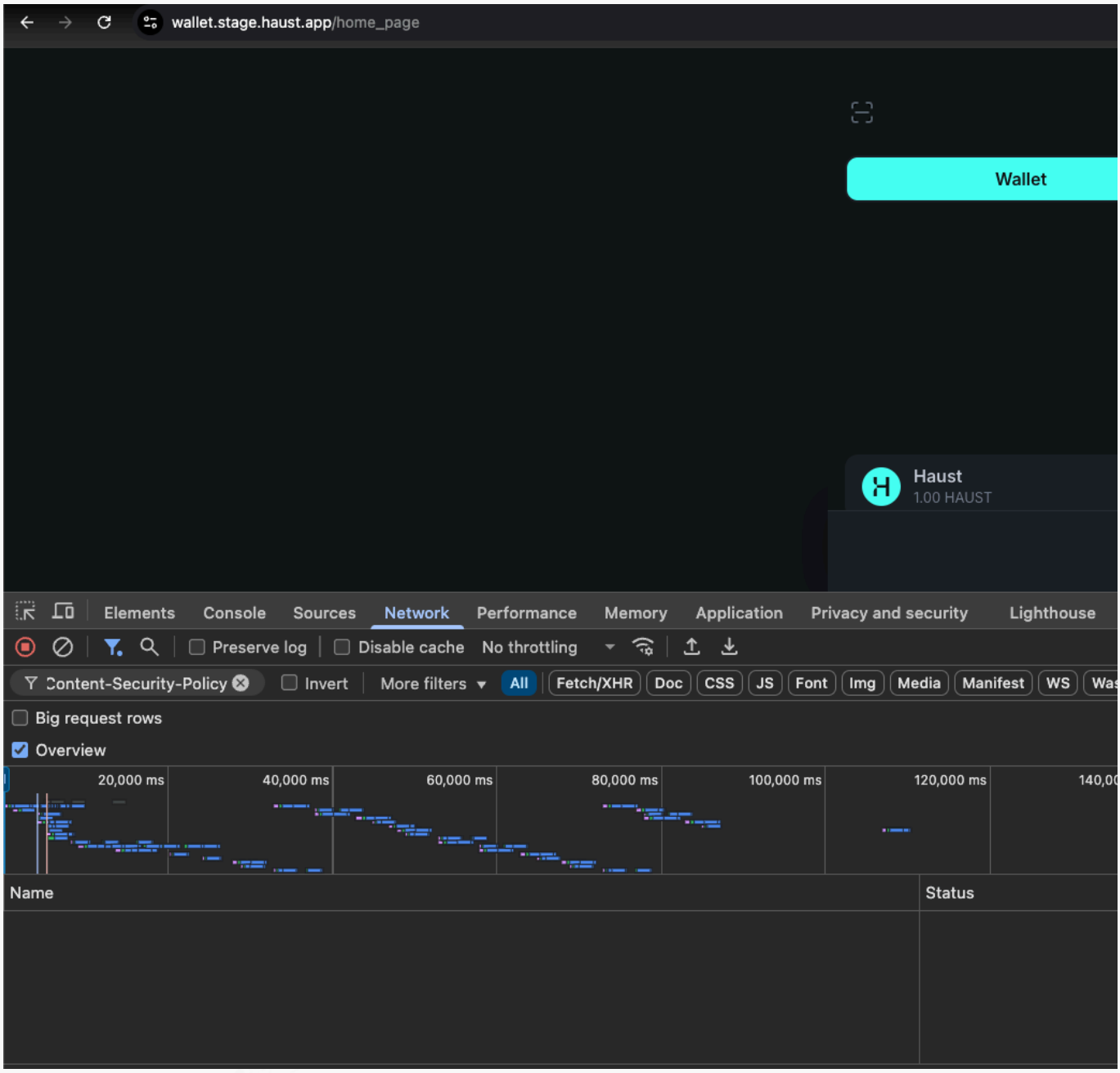
Fixed

[N4] [Suggestion] Lack of Content-Security-Policy security configuration

Category: HTTP response header security audit

Content

Content-Security-Policy is not configured.



Solution

It is recommended to configure "Content-Security-Policy". Content-Security-Policy (CSP) is a security mechanism used to restrict the resources that a browser can load from a website. It can help protect websites from cross-site scripting (XSS), data injection, and other security threats. If a website does not configure CSP, the browser will allow any resource to be loaded from any source. This makes it easier for attackers to exploit vulnerabilities in the website.

Status

Fixed

[N5] [Suggestion] Lack of X-Content-Type-Options security configuration

Category: HTTP response header security audit

Content

Check if the website has configured the 'x-content-type-options' security header.

```
[redacted] ~ % curl -s -I https://wallet.stage.haust.app/
HTTP/2 200
date: Thu, 03 Apr 2025 03:52:58 GMT
content-type: text/html
content-length: 1770
last-modified: Fri, 28 Mar 2025 08:03:51 GMT
etag: "67e657e7-6ea"
cache-control: no-store, no-cache, must-revalidate, max-age=0
pragma: no-cache
expires: 0
accept-ranges: bytes
strict-transport-security: max-age=31536000; includeSubDomains
```

Solution

Browser sniffing behavior can lead to risks such as a resource appearing to be an image but containing a string of text, such as `<script>alert(1);</script>`. This could potentially result in an XSS attack. While the occurrence of such attacks depends on specific scenarios and modern browsers have different strategies to address them, it is best security practice to completely eliminate this risk. HTTP response header configuration should include:

`X-Content-Type-Options: nosniff`

Status

Fixed

[N6] [Suggestion] Cross-domain transmission security issue

Category: Cross-domain transmission security audit

Content

The code uses `CorsLayer::permissive()` to allow requests from all sources, lacking necessary security restrictions.

Code location: haust-wallet-rs/crates/api/src/middleware.rs #L111-113

```
pub fn cors_layer() -> tower_http::cors::CorsLayer {
    tower_http::cors::CorsLayer::permissive()
}
```

Solution

It is recommended to explicitly specify the allowed domain name whitelist in the code.

Status

Fixed

[N7] [Low] WalletConnect issue

Category: Business design security audit

Content

When a wallet is linked to a decentralized application (DApp) via WalletConnect, the wallet does not display the connected DApp. Additionally, there is no specific location provided for disconnecting. As a result, users are unable to disconnect from the DApp through the wallet. In the event that users connect to a malicious DApp, the malicious DApp can trigger signatures at any time.

Solution

It is recommended to intuitively present the connection between the wallet and the DApp and enable users to manually disconnect.

Status

Fixed

[N8] [Suggestion] Email binding issue

Category: Business design security audit

Content

The user's wallet address is bound to the email address. Distinct email addresses cannot be bound to the same private key. Additionally, the wallet does not offer an email binding change interface. In the event that the user's email is lost or changed, it will result in the loss of the wallet.

Solution

It is recommended to incorporate an interface for modifying bound email addresses.

Status

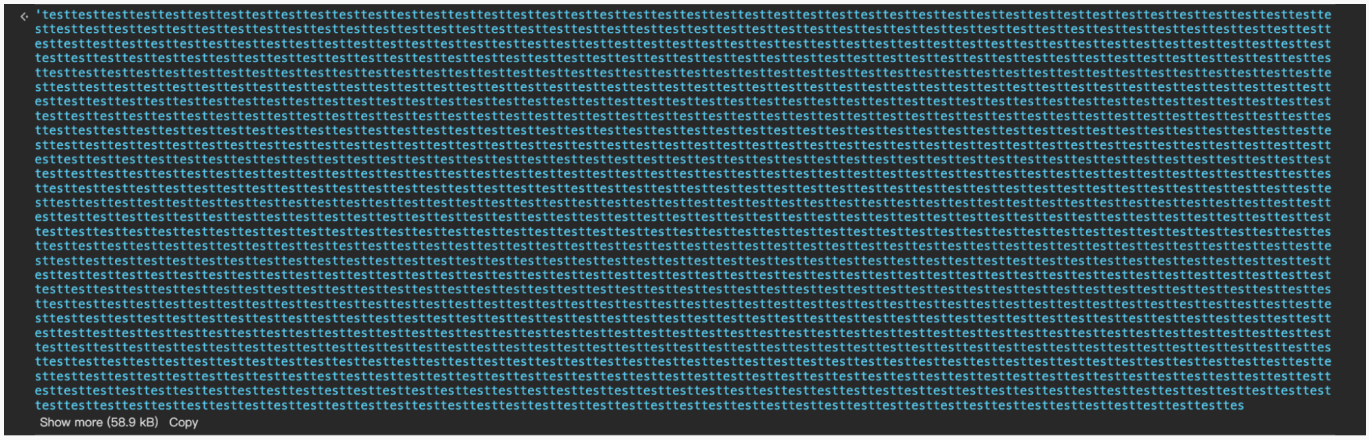
Fixed

[N9] [Low] Denial of Service security issue

Category: Denial of Service security audit

Content

A long signature will cause the wallet to crash. For example, initiate a signature using the following signature content:


'testtesttesttestt

Analyzing DNSSEC problems for wallet.stage.haust.app

.	✓ Found 4 DNSKEY records for . ✓ DS=20326/SHA-256 verifies DNSKEY=20326/SEP ✓ Found 1 RRSIGs over DNSKEY RRset ✓ RRSIG=20326 and DNSKEY=20326/SEP verifies the DNSKEY RRset
app	✓ Found 1 DS records for app in the . zone ✓ DS=23684/SHA-256 has algorithm RSASHA256 ✓ Found 1 RRSIGs over DS RRset ✓ RRSIG=53148 and DNSKEY=53148 verifies the DS RRset ✓ Found 2 DNSKEY records for app ✓ DS=23684/SHA-256 verifies DNSKEY=23684/SEP ✓ Found 1 RRSIGs over DNSKEY RRset ✓ RRSIG=23684 and DNSKEY=23684/SEP verifies the DNSKEY RRset
haust.app	✗ No DS records found for haust.app in the app zone ✗ No DNSKEY records found ✓ everton.ns.cloudflare.com is authoritative for wallet.stage.haust.app ✓ wallet.stage.haust.app A RR has value 151.115.12.0 ✗ No RRSIGs found
haust.app	✓ nola.ns.cloudflare.com is authoritative for wallet.stage.haust.app ✓ wallet.stage.haust.app A RR has value 151.115.12.0 ✗ No RRSIGs found

Move your mouse over any ✗ or ⚠ symbols for remediation hints.

Want a second opinion? Test wallet.stage.haust.app at dnsviz.net.

Solution

It is recommended to configure DNSSEC enabled.

Status

Fixed

[N11] [Suggestion] User interaction issue

Category: User interaction security

Content

Functionality	Support	Notes
WYSIWYS	✓	The current version only allows signature transfers via WalletConnect.
AML	✗	AML strategy is not supported.
Anti-phishing	✗	Phishing detect warning is not supported.

Functionality	Support	Notes
Pre-execution	✗	Pre-execution result display is not supported.
Contact whitelisting	✗	The contact whitelisting is not supported, causing similar address attacks.
Password complexity requirements	✗	There is no password complexity limit.

Tip: ✓ Full support, ● Partial support, ✗ No support

Solution

1. It is recommended to connect to AML.
2. It is recommended to check whether the domain name connected by wallet connect is a phishing domain name.
3. It is recommended to add the pre-execution function.
4. It is recommended to add the contact whitelist function.
5. It is recommended to limit the complexity of passwords.

Status

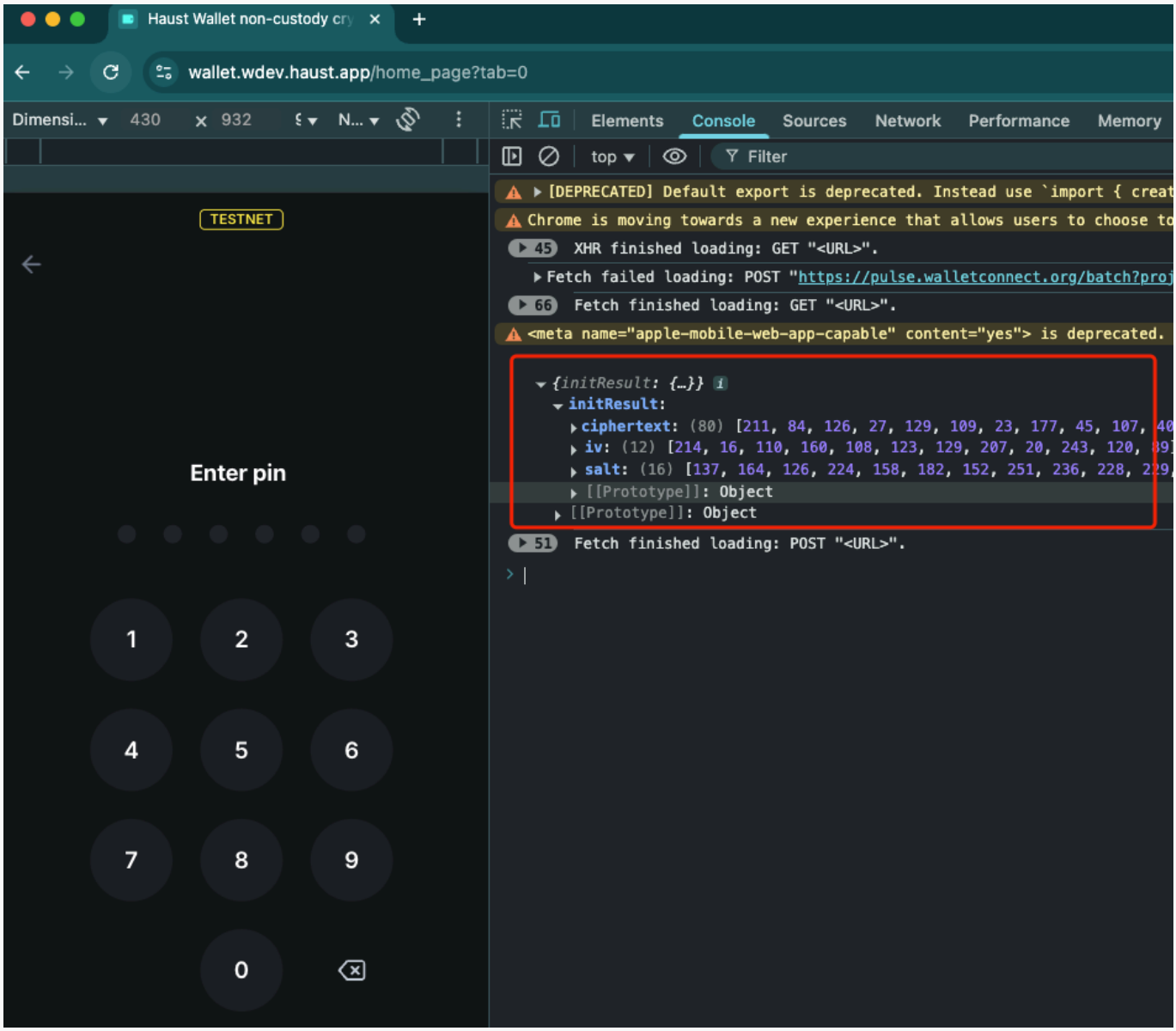
Acknowledged

[N12] [Suggestion] Debug information is printed to console log by default

Category: Others

Content

Refresh the webpage, open the browser console and you can see that it defaults to outputting an initResult data, which contains encrypted private key information, possibly because the project test Debug did not remove the printout.



Solution

It is recommended to remove such high-risk debugging information in the production environment.

Status

Fixed

[N13] [Low] API error exposes detailed stack trace leading to technology stack disclosure

Category: Others

Content

In the application error handling process, when an exception occurs, complete stack trace information is directly returned to the end user. This trace information contains sensitive details about the application architecture, frameworks used, middleware structure, and internal error handling logic.

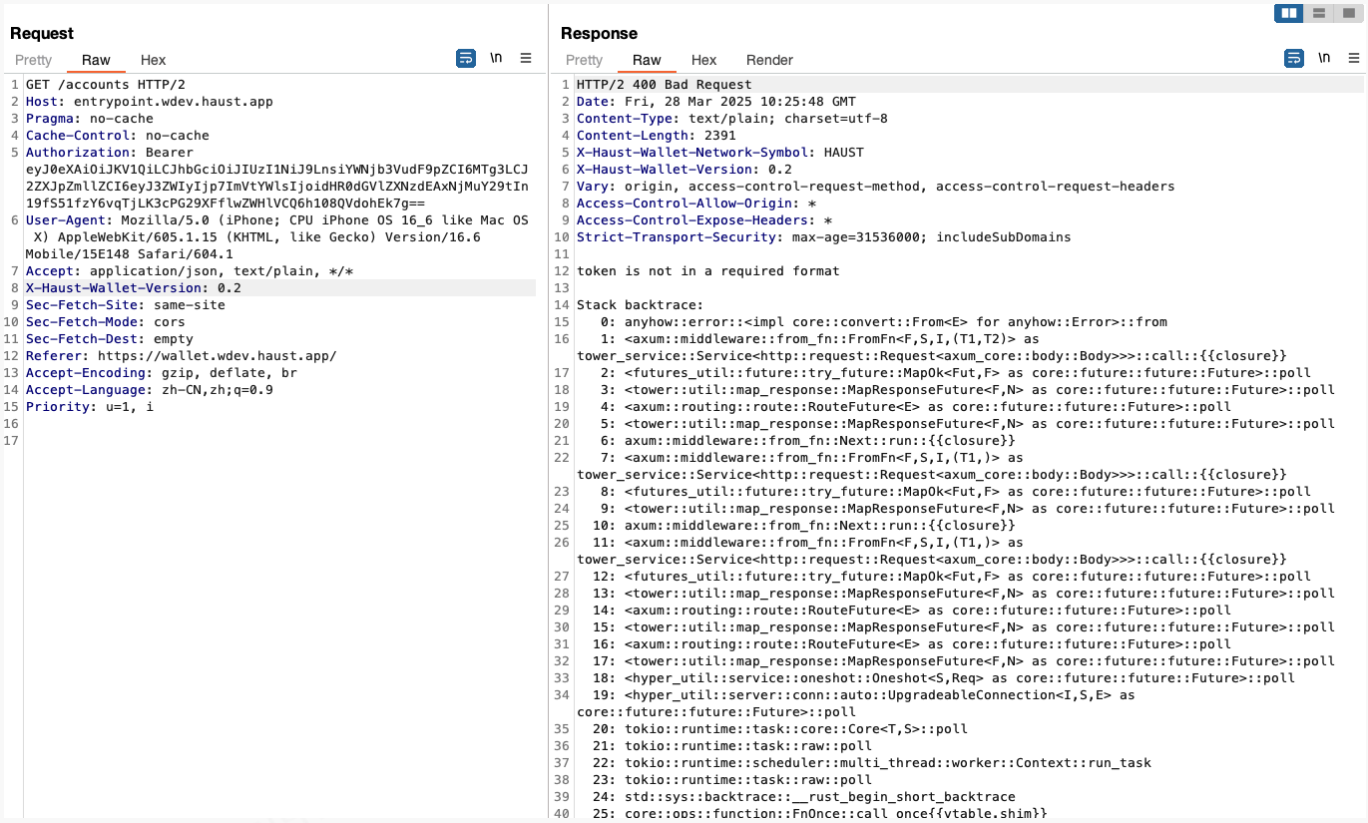
The specific information leaked includes:

- Details of the technology stack used by the application (Axum, Tokio, Hyper, Tower)
- Middleware call chain and routing structure
- Internal error handling mechanisms and call hierarchy
- Server architecture (multi-threaded Tokio runtime)

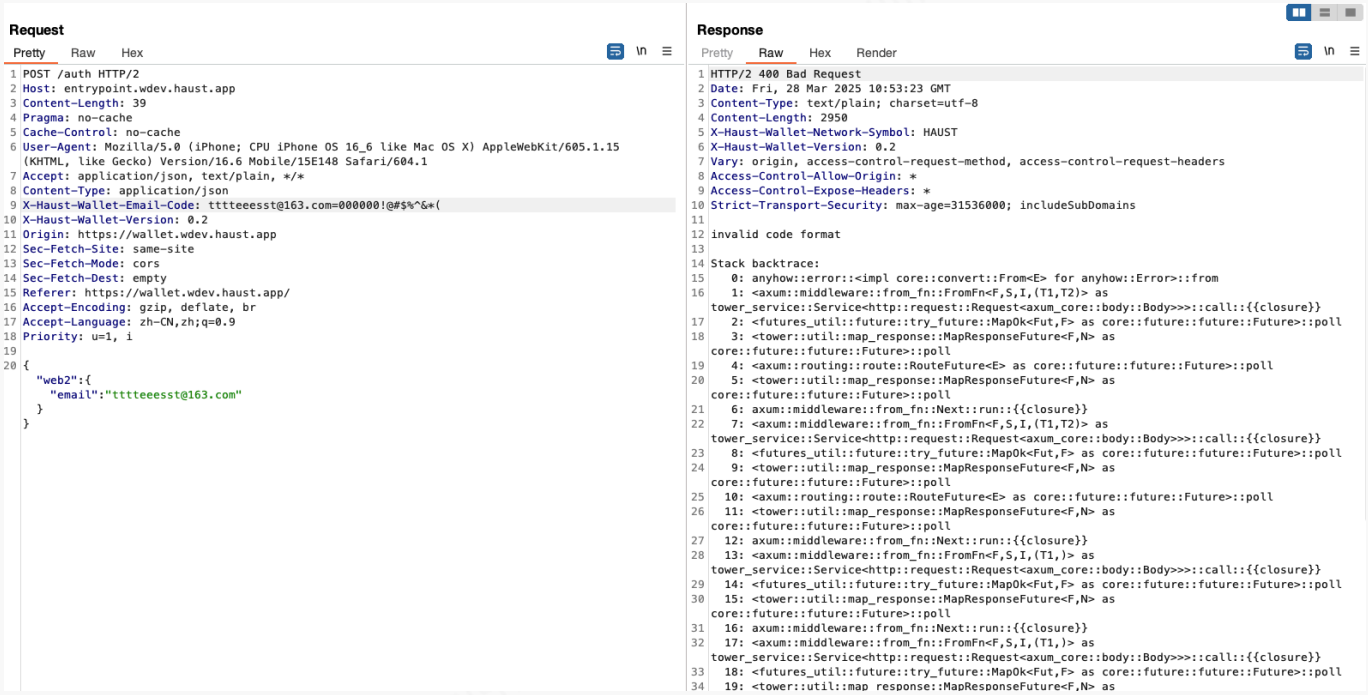
Example stack trace:

When an Authorization that does not comply with the JWT standard is provided, it triggers an error in the interface and returns the complete stack trace information of the error.

```
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9LnsiYWNjb3VudF9pZCI6MTg3LCJ2ZXJpZmllZCI6eyJ3ZWIyIjp7Im
VtYWlsIjoidGVzdEB0ZXN0LmNvbSJ9fX0udX82Or6k4yyt3DxtvVxX5cGVh5VQkOoddPEFXaIRJO4
```



example 2:



Request

Pretty Raw Hex

```
POST /auth HTTP/2
Host: entrypoint.wdev.haust.app
Content-Length: 39
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 16_6 like Mac OS X) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/16.6 Mobile/15E148 Safari/604.1
Accept: application/json, text/plain, */*
Content-Type: application/json
X-Haust-Wallet-Email-Code: ttteeeesst@163.com=000000!@#$%^&*(
X-Haust-Wallet-Version: 0.2
Origin: https://wallet.wdev.haust.app
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://wallet.wdev.haust.app/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Priority: u=1, i

{
  "web2":{
    "email":"ttteeeesst@163.com"
  }
}
```

Response

Pretty Raw Hex Render

```
HTTP/2 400 Bad Request
Date: Fri, 28 Mar 2025 10:53:23 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 2950
X-Haust-Wallet-Network-Symbol: HAUST
X-Haust-Wallet-Version: 0.2
Vary: origin, access-control-request-method, access-control-request-headers
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: *
Strict-Transport-Security: max-age=31536000; includeSubDomains

invalid code format

Stack backtrace:
   0: anyhow::error::<impl core::convert::From<E> for anyhow::Error>::from
   1: <axum::middleware::from_fn::FromFn<F,S,I,(T1,T2)> as
tower_service::Service<http::request::Request<axum_core::body::Body>>>::call::{{closure}}
   2: <futures_util::future::try_future::MapOk<Fut,F> as core::future::future::Future>::poll
   3: <tower::util::map_response::MapResponseFuture<F,N> as
core::future::future::Future>::poll
   4: <axum::routing::route::RouteFuture<E> as core::future::future::Future>::poll
   5: <tower::util::map_response::MapResponseFuture<F,N> as
core::future::future::Future>::poll
   6: axum::middleware::from_fn::Next::run::{{closure}}
   7: <axum::middleware::from_fn::FromFn<F,S,I,(T1,T2)> as
tower_service::Service<http::request::Request<axum_core::body::Body>>>::call::{{closure}}
   8: <futures_util::future::try_future::MapOk<Fut,F> as core::future::future::Future>::poll
   9: <tower::util::map_response::MapResponseFuture<F,N> as
core::future::future::Future>::poll
  10: <axum::routing::route::RouteFuture<E> as core::future::future::Future>::poll
  11: <tower::util::map_response::MapResponseFuture<F,N> as
core::future::future::Future>::poll
  12: axum::middleware::from_fn::Next::run::{{closure}}
  13: <axum::middleware::from_fn::FromFn<F,S,I,(T1,)> as
tower_service::Service<http::request::Request<axum_core::body::Body>>>::call::{{closure}}
  14: <futures_util::future::try_future::MapOk<Fut,F> as core::future::future::Future>::poll
  15: <tower::util::map_response::MapResponseFuture<F,N> as
core::future::future::Future>::poll
  16: axum::middleware::from_fn::Next::run::{{closure}}
  17: <axum::middleware::from_fn::FromFn<F,S,I,(T1,)> as
tower_service::Service<http::request::Request<axum_core::body::Body>>>::call::{{closure}}
  18: <futures_util::future::try_future::MapOk<Fut,F> as core::future::future::Future>::poll
  19: <tower::util::map_response::MapResponseFuture<F,N> as
```

Solution

It is recommended to standardize the error message specifications to avoid directly exposing the backend error messages to the frontend.

Status

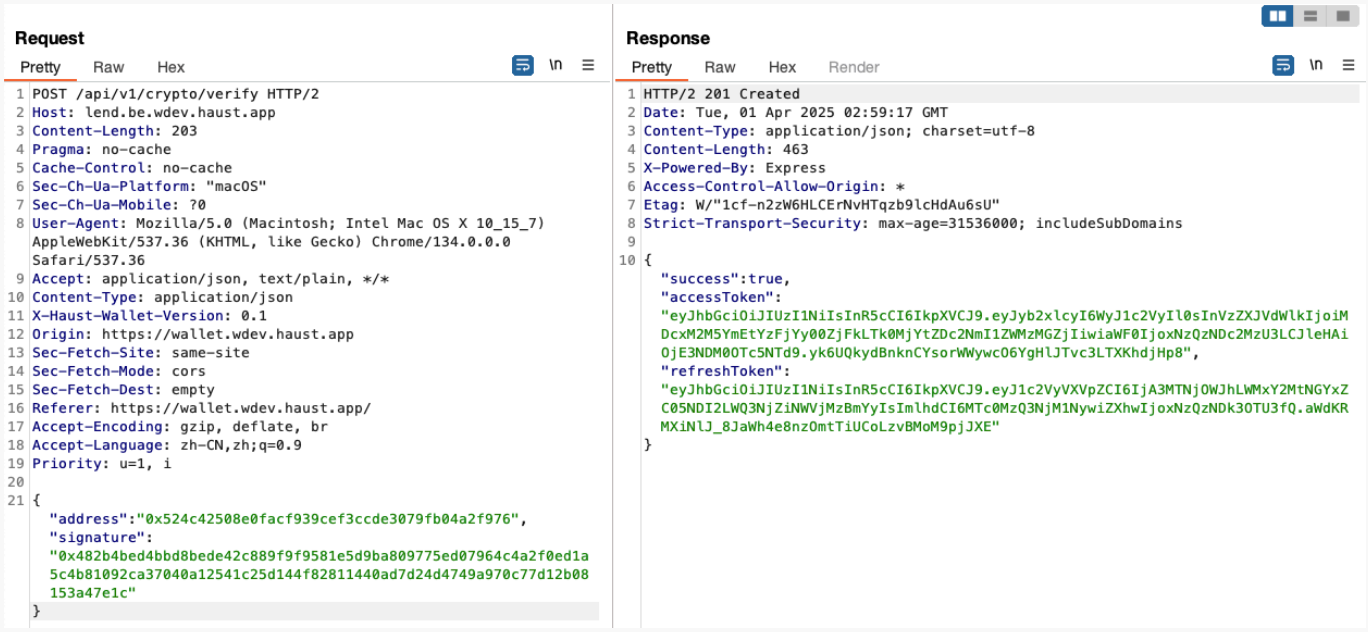
Fixed

[N14] [Medium] Signatures can be reused multiple times

Category: Others

Content

The endpoint https://lend.be.wdev.haust.app/api/v1/crypto/verify is used to verify the validity of the signature. Upon successful validation, the server issues an access token and refresh token.



Request

Pretty Raw Hex

```
POST /api/v1/crypto/verify HTTP/2
Host: lend.be.wdev.haust.app
Content-Length: 203
Pragma: no-cache
Cache-Control: no-cache
Sec-Ch-Ua-Platform: "macOS"
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0
Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
X-Haust-Wallet-Version: 0.1
Origin: https://wallet.wdev.haust.app
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://wallet.wdev.haust.app/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Priority: u=1, i

{
  "address":"0x524c42508e0facf939cef3ccde3079fb04a2f976",
  "signature":
  "0x482b4bed4bbd8bede42c889f9f9581e5d9ba809775ed07964c4a2f0ed1a
  5c4b81092ca37040a12541c25d144f82811440ad7d24d4749a970c77d12b08
  153a47e1c"
}
```

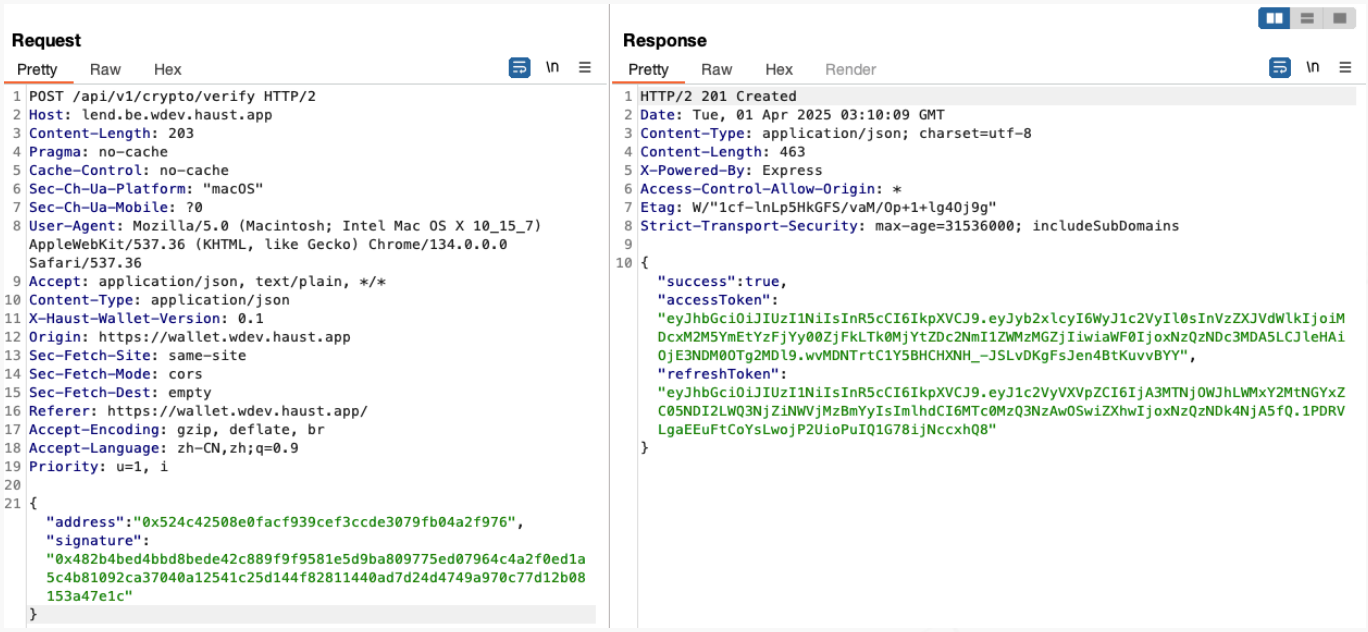
Response

Pretty Raw Hex Render

```
HTTP/2 201 Created
Date: Tue, 01 Apr 2025 02:59:17 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 463
X-Powered-By: Express
Access-Control-Allow-Origin: *
Etag: W/"1cf-n2zW6HLCErNvHTqzb9lcHdAu6sU"
Strict-Transport-Security: max-age=31536000; includeSubDomains

{
  "success":true,
  "accessToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlcyI6WyJ1c2VyIl0sInVzZXJVdWlkIjoiM
  DcxM2M5YmEtYzFjYy00ZjFkLTk0MjYtZDc2NmI1ZWMzMGZjIiwiaWF0IjoxNzQzNDc2MzU3LCJleHAi
  OjE3NDM0OTc5NTd9.yk6UQkydBnknCYsorWWywcO6YgHlJTvc3LTXKhdjHp8",
  "refreshToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyVXVpZCI6IjA3MTNjOWJhLWMxY2MtNGYxZ
  C05NDI2LWQ3NjZiNWVjMzBmYyIsImlhdCI6MTc0MzQ3NjM1NywiZXhwIjoxNzQzNDk3OTU3fQ.aWdKR
  MXiNlJ_8JaWh4e8nzOmtTiUCoLzvBMoM9pjJXE"
}
```

After initially obtaining a token, the same signature can be used repeatedly to obtain new tokens. The signature's validity is not restricted to a single use but remains effective for multiple authentications over an extended period.



Request

Pretty Raw Hex

```
POST /api/v1/crypto/verify HTTP/2
Host: lend.be.wdev.haust.app
Content-Length: 203
Pragma: no-cache
Cache-Control: no-cache
Sec-Ch-Ua-Platform: "macOS"
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0
Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
X-Haust-Wallet-Version: 0.1
Origin: https://wallet.wdev.haust.app
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://wallet.wdev.haust.app/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Priority: u=1, i

{
  "address":"0x524c42508e0facf939cef3ccde3079fb04a2f976",
  "signature":
  "0x482b4bed4bbd8bede42c889f9f9581e5d9ba809775ed07964c4a2f0ed1a
  5c4b81092ca37040a12541c25d144f82811440ad7d24d4749a970c77d12b08
  153a47e1c"
}
```

Response

Pretty Raw Hex Render

```
HTTP/2 201 Created
Date: Tue, 01 Apr 2025 03:10:09 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 463
X-Powered-By: Express
Access-Control-Allow-Origin: *
Etag: W/"1cf-lnLp5HkGFS/vaM/Op+1+lg4Oj9g"
Strict-Transport-Security: max-age=31536000; includeSubDomains

{
  "success":true,
  "accessToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlcyI6WyJ1c2VyIl0sInVzZXJVdWlkIjoiM
  DcxM2M5YmEtYzFjYy00ZjFkLTk0MjYtZDc2NmI1ZWMzMGZjIiwiaWF0IjoxNzQzNDc3MDA5LCJleHAi
  OjE3NDM0OTg2MDl9.wvMDNTrtC1Y5BHCHXNH_-JSLvDKgFsJen4BtKuvvBYY",
  "refreshToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyVXVpZCI6IjA3MTNjOWJhLWMxY2MtNGYxZ
  C05NDI2LWQ3NjZiNWVjMzBmYyIsImlhdCI6MTc0MzQ3NzAwOSwiZXhwIjoxNzQzNDk4NjA5fQ.1PDRV
  LgaEEuFtCoYsLwojP2UioPuIQ1G78ijNccxhQ8"
}
```

Solution

It is recommended to implement verification to check if a signature has been previously used, enforcing a single-use policy and limiting its validity period.

Status

Fixed

[N15] [Low] Insufficient authentication security due to missing JWT claims

Category: Others

Content

The current JWT token structure lacks necessary security fields, resulting in inadequate token security. Specifically, several critical JWT claims are missing, which are essential for ensuring the security and validity of the token.

Missing exp (expiration time) field: This causes tokens to remain valid indefinitely, allowing attackers to use stolen tokens permanently.

Missing iat (issued at time) field: This makes it impossible to track the token lifecycle, hindering token rotation strategies.

Missing jti (JWT ID) field: This prevents implementation of token blacklisting mechanisms, making effective token revocation impossible.

Missing aud (audience) and iss (issuer) fields: This prevents verification of token source and purpose, providing no protection against token replay attacks.

Code location: crates/api/src/middleware/auth_token.rs # L24-57

```
impl AuthToken {
  /// Encode [`AuthToken`] in jwt claims for use in [`Authorization<Bearer>`] header.
  fn encode_internal(&self, jwt: &Jwt, secret: &jwt::Secret) -> crate::Result<String>
{
      // TODO: add `exp`, etc.
      jwt.encode(secret, self).map_err(Into::into)
  }

  /// Encode [`AuthToken`] in jwt claims for use in [`Authorization<Bearer>`] header.
  pub fn encode(&self, st: &Shared) -> crate::Result<String> {
      // TODO: add `exp`, etc.
      self.encode_internal(&st.jwt.1, &st.jwt.0)
  }

  /// Decode [`AuthToken`] from jwt claims in [`Authorization<Bearer>`] header.
  ///
  /// Inserts [`AuthToken`] as request extension.
  pub async fn decoding_layer(
      State(st): State<Shared>,
      mut req: Request,
      next: Next,
  ) -> crate::Result<Response> {
      let Some(auth) = req
          .headers()
          .typed_get::<Authorization<authorization::Bearer>>()
```

```
        else {
            return Ok(next.run(req).await);
        };
        let (jwt_secret, jwt) = st.jwt.as_ref();
        let decoded: AuthToken = jwt.decode(jwt_secret, auth.token())?;
        tracing::trace!(?decoded, "auth token");
        req.extensions_mut().insert(decoded);
        Ok(next.run(req).await)
    }
}
```

Solution

It is recommended to add the missing JWT Claims.

Status

Fixed

[N16] [Low] Incomplete token validation

Category: Others

Content

The JWT token validation process is incomplete, with multiple security validations missing. The system fails to verify whether the token has expired during the decoding process, and also does not perform necessary validation of the token's signature algorithm. Additionally, the system does not check the audience and issuer information of the token, and lacks an effective token blacklist checking mechanism, all of which lead to the incompleteness of the validation process.

Code location: crates/api/src/middleware/auth_token.rs # L24-57

```
impl AuthToken {
    /// Encode [`AuthToken`] in jwt claims for use in [`Authorization<Bearer>`] header.
    fn encode_internal(&self, jwt: &Jwt, secret: &jwt::Secret) -> crate::Result<String>
{
        // TODO: add `exp`, etc.
        jwt.encode(secret, self).map_err(Into::into)
    }

    /// Encode [`AuthToken`] in jwt claims for use in [`Authorization<Bearer>`] header.
    pub fn encode(&self, st: &Shared) -> crate::Result<String> {
        // TODO: add `exp`, etc.
        self.encode_internal(&st.jwt.1, &st.jwt.0)
    }
```

```
    /// Decode [`AuthToken`] from jwt claims in [`Authorization<Bearer>`] header.
    ///
    /// Inserts [`AuthToken`] as request extension.
    pub async fn decoding_layer(
        State(st): State<Shared>,
        mut req: Request,
        next: Next,
    ) -> crate::Result<Response> {
        let Some(auth) = req
            .headers()
            .typed_get::<Authorization<authorization::Bearer>>()
        else {
            return Ok(next.run(req).await);
        };
        let (jwt_secret, jwt) = st.jwt.as_ref();
        let decoded: AuthToken = jwt.decode(jwt_secret, auth.token())?;
        tracing::trace!(?decoded, "auth token");
        req.extensions_mut().insert(decoded);
        Ok(next.run(req).await)
    }
}
```

Solution

It is recommended to implement a comprehensive token validation process that includes expiration verification, signature algorithm validation, audience and issuer checks, and a token blacklist mechanism. This will significantly improve the security of the authentication system and reduce the risk of unauthorized access through compromised tokens.

Status

Fixed

[N17] [Low] Arbitrary email spoofing due to SPF configuration errors

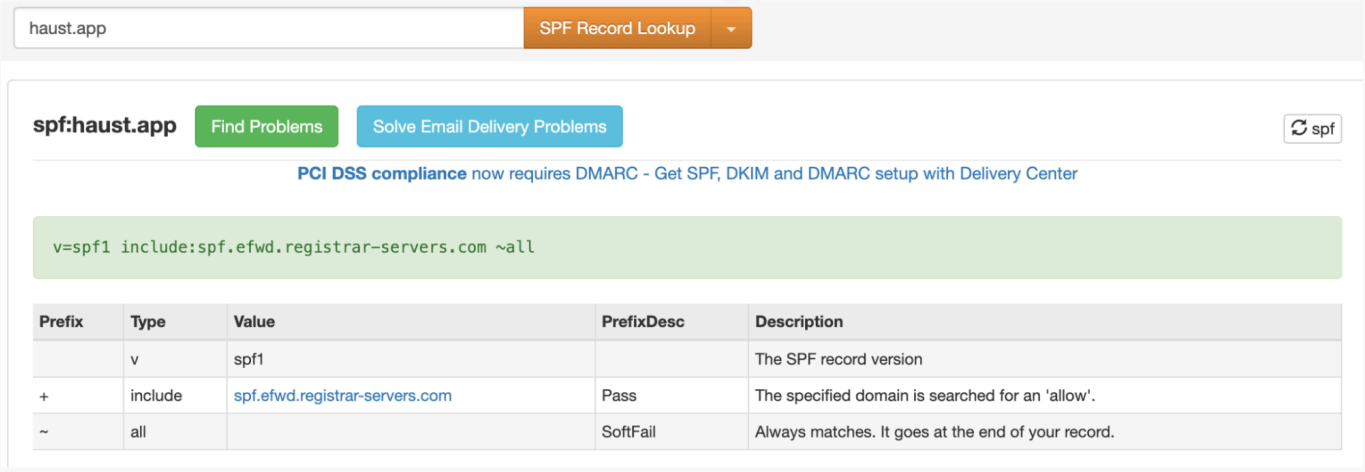
Category: Others

Content

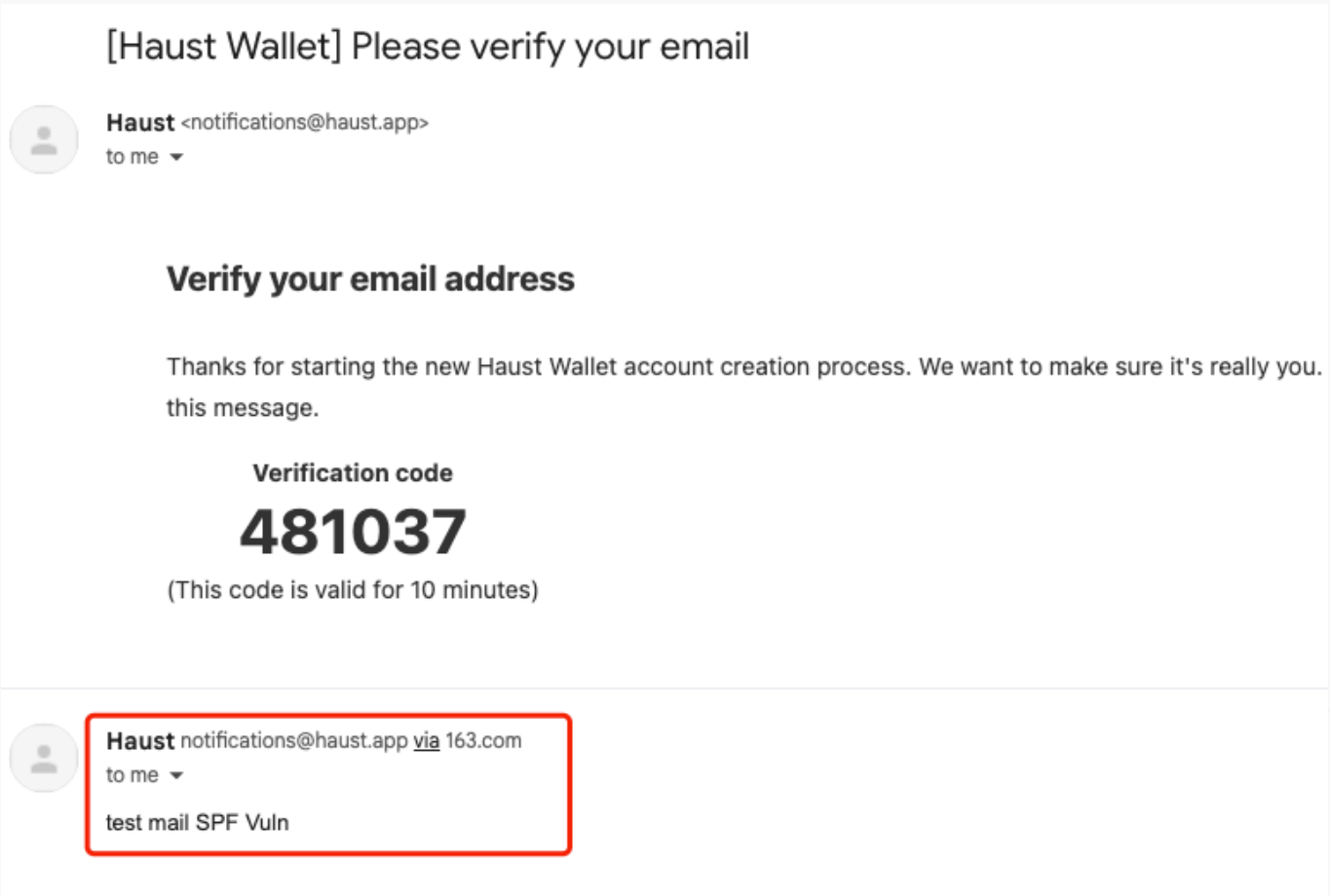
The SPF record configuration for the domain haust.app has security flaws, currently using a soft fail mechanism (`~all`) instead of a hard fail mechanism (`-all`), while DMARC policy enforcement is not enabled. This allows attackers to forge emails from this domain, bypassing email authentication mechanisms.

Testing has confirmed that it is possible to send spoofed emails through third-party mail servers (such as 163.com)

that appear to come from the haust.app domain. The current SPF record is configured as `v=spf1 include:spf.efwd.registrar-servers.com ~all`, which uses soft fail mode. Additionally, DMARC has not enabled Quarantine or Reject policies, preventing effective interception of emails that violate SPF rules. This enables attackers to use third-party mail servers to send phishing emails to target users, impersonating official notifications from haust.app.



(Reffer: https://mxtoolbox.com/SuperTool.aspx?action=mx:haust.app&run=toolpage)



Solution

It is recommended to modify the SPF record from `~all` to `-all` to enable hard fail mechanism, explicitly rejecting

unauthorized sending sources. Implement DKIM email signing mechanism to add cryptographic signatures to all outbound emails. Configure mandatory DMARC policy, starting with `p=none` to monitor reports, then upgrading to `p=quarantine` after confirming legitimate emails are not affected, and finally setting to `p=reject` to completely reject non-compliant emails.

Status

Fixed

[N18] [Suggestion] Lack of anti-phishing security code

Category: Others

Content

The wallet shall be bound to the user's email address. During registration/login, verification codes will be dispatched to the user's email address. The sent emails currently lack anti-phishing security strategies.

Solution

It is recommended to incorporate anti-phishing security codes to assist users in determining the authenticity of the email source.

Status

Fixed

4 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002504100001	SlowMist Security Team	2025.03.27 - 2025.04.10	Passed

Summary conclusion: The SlowMist security team employs a manual approach in combination with the analysis tool developed by the SlowMist team to conduct an audit of the project. During the entire audit process, one high-risk issue, two medium-risk issues, six low-risk issues, and nine suggestions were detected. One suggestion has been acknowledged. All the other findings have been fixed.

5 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website

www.slowmist.com



E-mail

team@slowmist.com



Twitter

@SlowMist_Team



Github

https://github.com/slowmist